

# Lab 10 — Create Custom Codex Skills

**Course:** Codex Masterclass (C989)

**Topic 3** · about 25 minutes · slides 88–93

**Surface:** Codex

**Features:** \$skill-installer · \$skill-creator · implicit triggering

## The story so far

Mid-Autumn is coming and Grace wants a Mooncake Making course on the site next week. Adding a course touches the catalogue, a brochure, the knowledge base and a test — every season. Do it once by hand, then make it a skill anyone can run.

## Your goal

Adding a course touches four files and a test. Do it once by hand, then save it as a skill anyone can run.

## You'll build

\$kb-update and \$course-brochure skills, and a new course live

## What is in this folder

- assets/BAK-111-mooncake.md
- assets/project-setup-skill/
- assets/skill-reference.md
- solution/ — reference files from the verified build
- prompts.md / prompts.pdf — every prompt, ready to paste
- evidence/checklist.md — what to capture as proof

## Step by step

1. **Install a given skill** — Paste Prompt A — install only, do not run.
2. **Do the job by hand** — From BAK-111-mooncake.md, add a row to your project's data/courses.csv (from Lab 2), a brochure and a golden question. Run npm run check.
3. **Save it as a skill** — Paste Prompt B.
4. **Create a second skill** — Paste Prompt C for course-brochure.
5. **Test by name** — Run \$course-brochure BAK-111.
6. **Test the trigger** — In a NEW chat type: "we are adding a Pineapple Tart course". \$kb-update should fire on its own.

## The prompts

### PROMPT A — Codex

Use \$skill-installer to install the attached `project-setup` skill.

Do not run the skill or build the project yet. Confirm the exact installed path and tell me when `project-setup` appears in this project's skill list. Do not write outside this project folder.

### PROMPT B — Codex

\$skill-creator Save what we just did as a project skill called `kb-update`.

It should trigger when a course is added, changed or withdrawn, or a fee, date, allergen or policy changes. Steps: update `data/courses.csv` first, then the `kb/` brochure to match, then run `npm run check`. Never edit the golden questions to pass. Report files changed and the eval score.

### PROMPT C — Codex

\$skill-creator Create a project skill called `course-brochure`: given a course code, write a one-page A4 HTML brochure from `data/courses.csv` and the `kb/` brochure only — photo, schedule, intakes, fee, what you learn, allergens, sign-up link. Stop and report if the two sources disagree.

## Check your work

- ☐ `project-setup` is installed and listed, but was not run.
- ☐ BAK-111 is on the site and `npm run check` passes 31/31.
- ☐ `kb-update` and `course-brochure` each have a "Use when" description.
- ☐ \$`course-brochure` BAK-111 produced a one-page A4 brochure.
- ☐ \$`kb-update` fired on its own in a new chat.

## If it goes wrong

- **The skill never fires** — Its description is too vague. Rewrite it with the trigger words and start a new chat.

## Stretch

- Withdraw BAK-109 with \$`kb-update` and confirm it marks it withdrawn rather than deleting it.
- Save your Lab 4 publish routine as a \$`gitpush` skill (see `solution/.agents/skills/gitpush`).

**Why it matters:** The description is the trigger. Write "Use when...", naming the words a colleague would actually type.

## Next

Lab 11 — A Workshop Popup and a Hook That Checks Every Edit. Keep what you built — the next lab starts from it.

## Safety

Use only this lab's synthetic data and your own accounts. Never paste a real API key, password or personal data into a prompt or a file. Every email address in the data must resolve to your own inbox. Nothing is published or sent without your explicit approval.